

Recursion ve Pointerlar



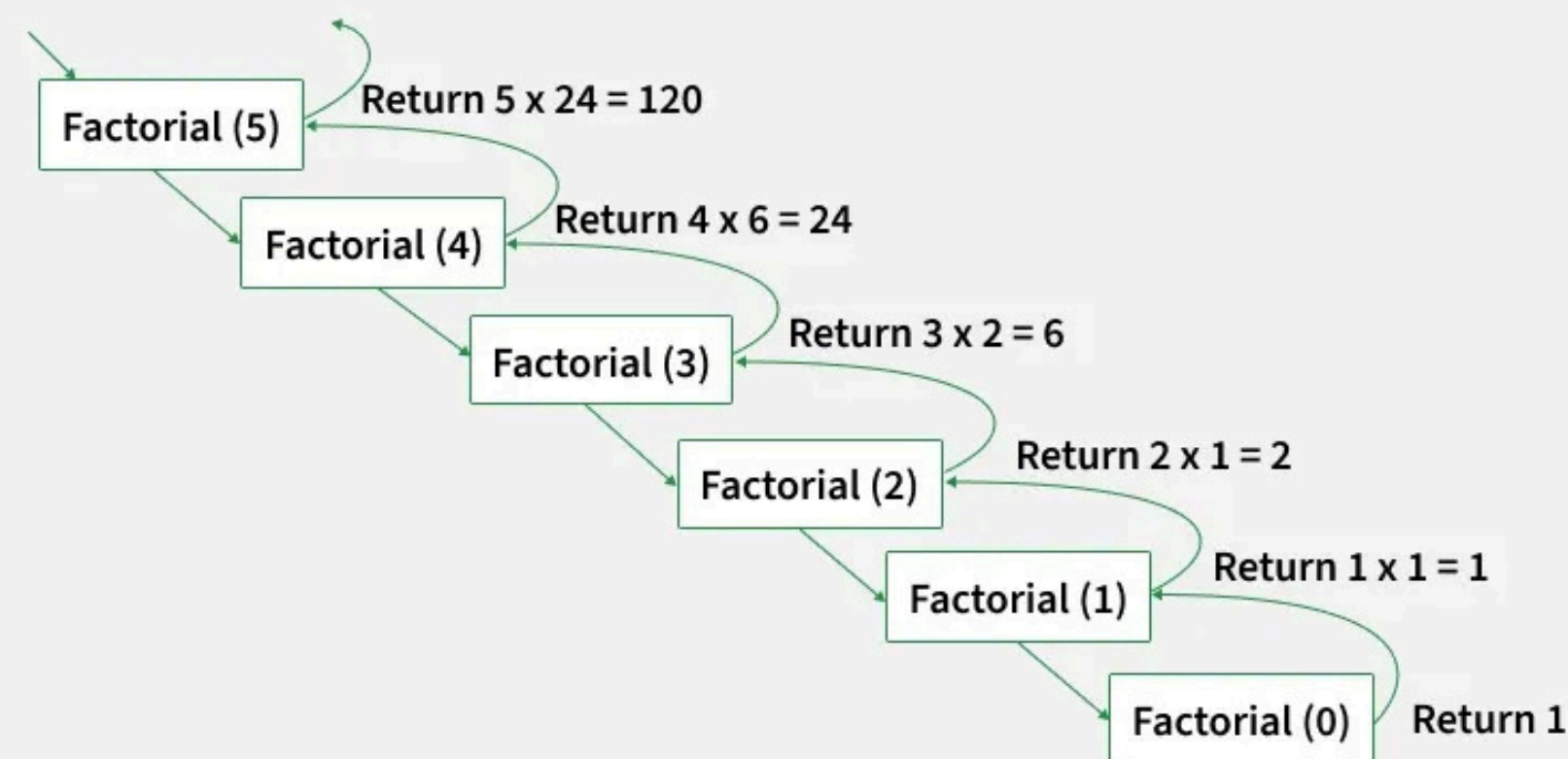
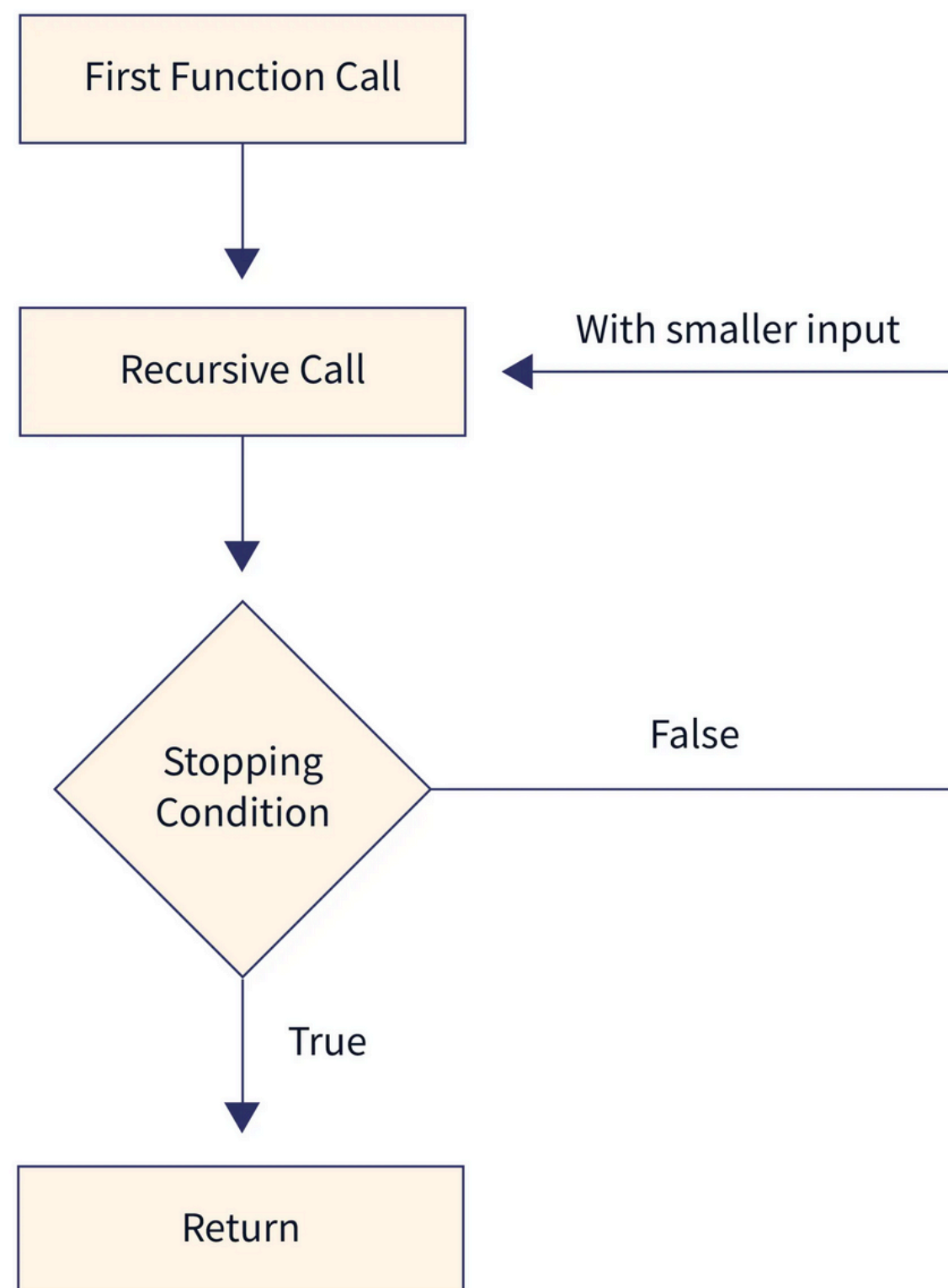
Kaan Yazıcı



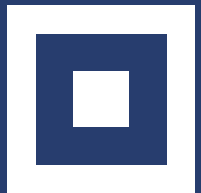
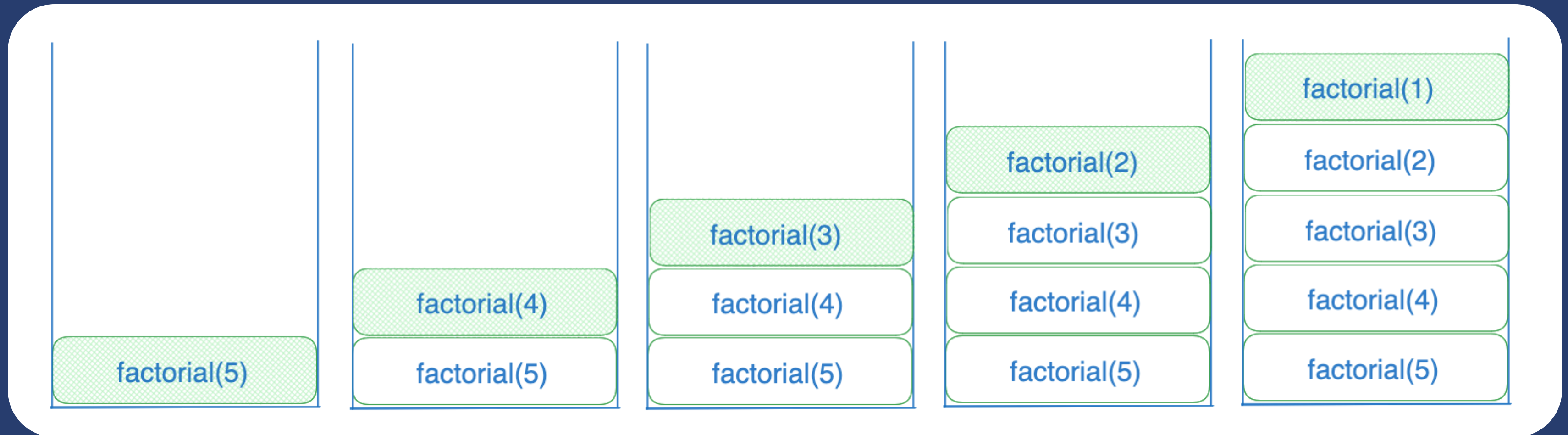
Recursion'a Giriş

Bir işlemin kendisini veya başka bir işlemi tekrarlı olarak çağırmasına **recursion** (*özyineleme*) adı verilir.

Bu özelliği gösteren fonksiyonlara **recursive function** (*özyinelemeli fonksiyon*) denir.



Call Stack kavramı



```
void iterative_dfs(const TreeNode* root) {  
    if (root == nullptr) {  
        return;  
    }  
  
    std::stack<const TreeNode*> s{};  
    s.push(root);  
  
    while (!s.empty()) {  
        const TreeNode* node{ s.top() };  
        s.pop();  
  
        std::cout << node->val << ' ';  
        if (node->left) {  
            s.push(node->left);  
        }  
        if (node->right) {  
            s.push(node->right);  
        }  
    }  
}
```

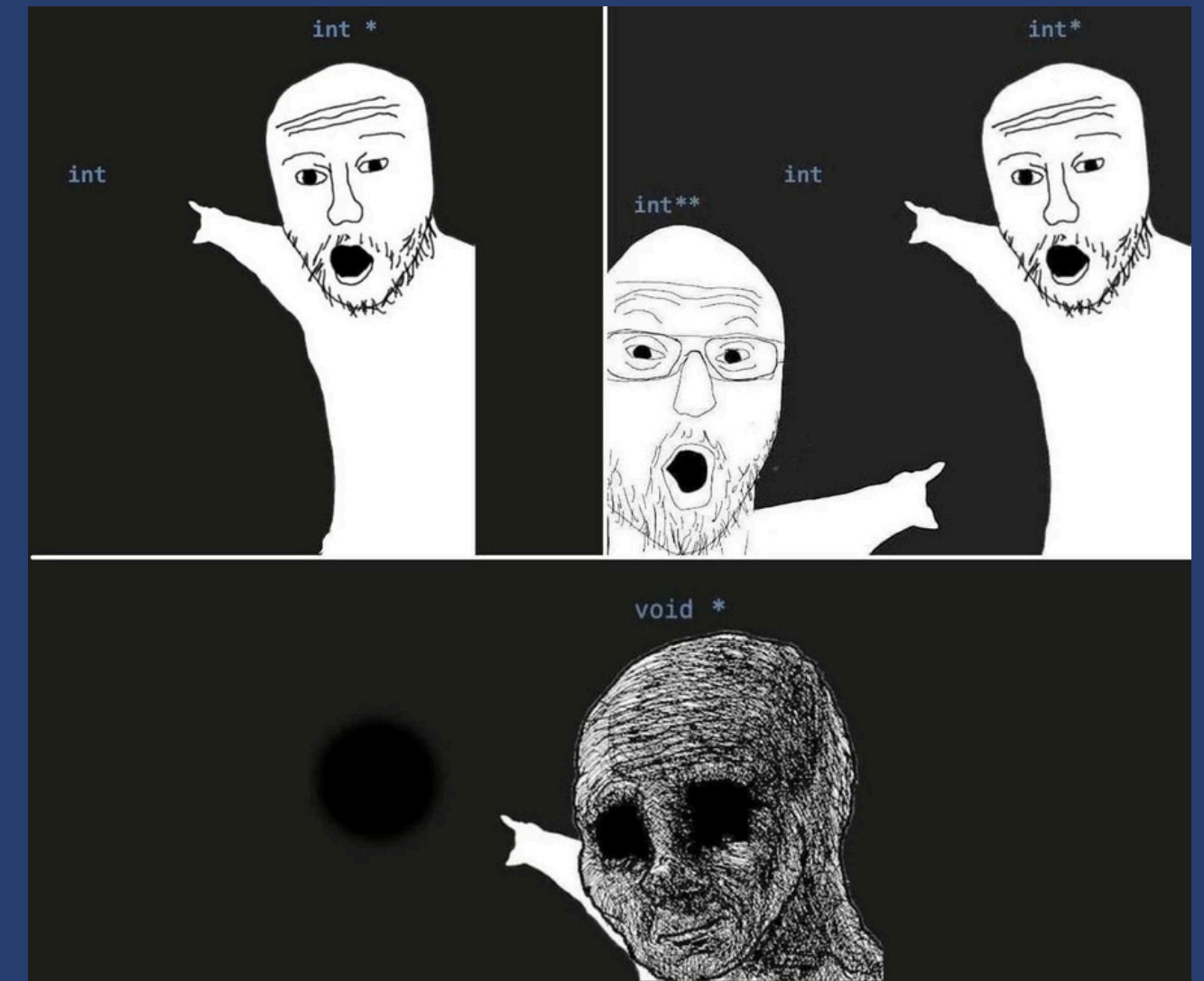
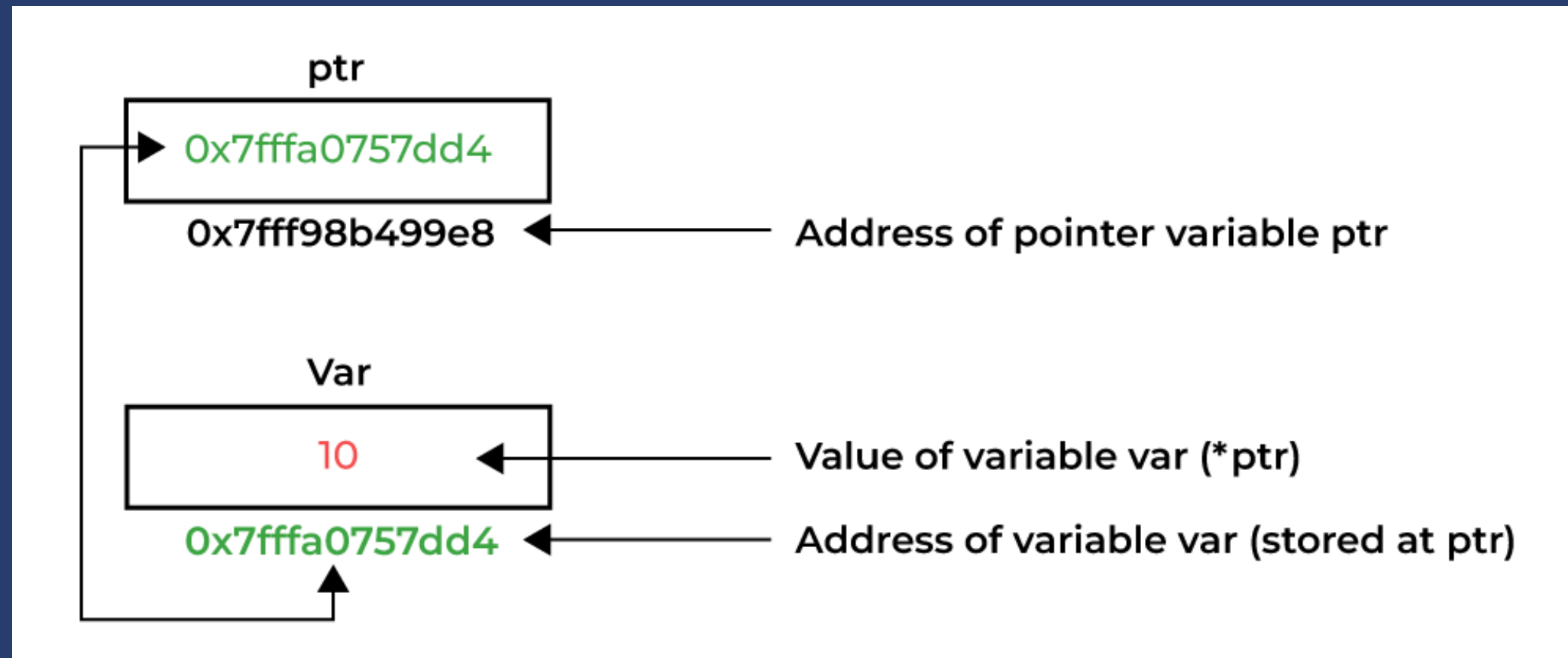


```
void recursive_dfs(const TreeNode* root) {  
    if (root == nullptr) {  
        return;  
    }  
    std::cout << root->val << ' ';  
    recursive_dfs(root->left);  
    recursive_dfs(root->right);  
}
```

Pointerlar



Değer olarak bir bellek adresi tutan değişkenlerdir.
Bellekte *hexadecimal* formatta yer tutarlar ve boyutları sistem içinde sabit olup sistemler arası değişiklik gösterebilir.



Stack ve Heap



Stack belleğinde, yerel değişkenler, fonksiyon çağrıları yer alır.
Bellek ayrımı ve temizliği program tarafından yapılır.
Buradaki değişkenler geçici yaşam süresine sahiptir.

```
int x;  
double pi = 3.14;  
char arr[3] = {'a', 'b', 'c'};  
void foo(int a, int b) {}
```

Stack ve Heap devam



Heap belleğinde, belleği dinamik şekilde ayrılmış değişkenler yer alır.

Bütün yönetimi programcı yapar.

Yaşam süresi, el ile temizlenene kadar devam eder.

Temizlenmezse, **memory leak** (*bellek sızıntısı*) görülebilir.

```
int* p = new int(10);  
int* ptr_to_arr = new int[5];  
MyClass* cls = new MyClass();
```

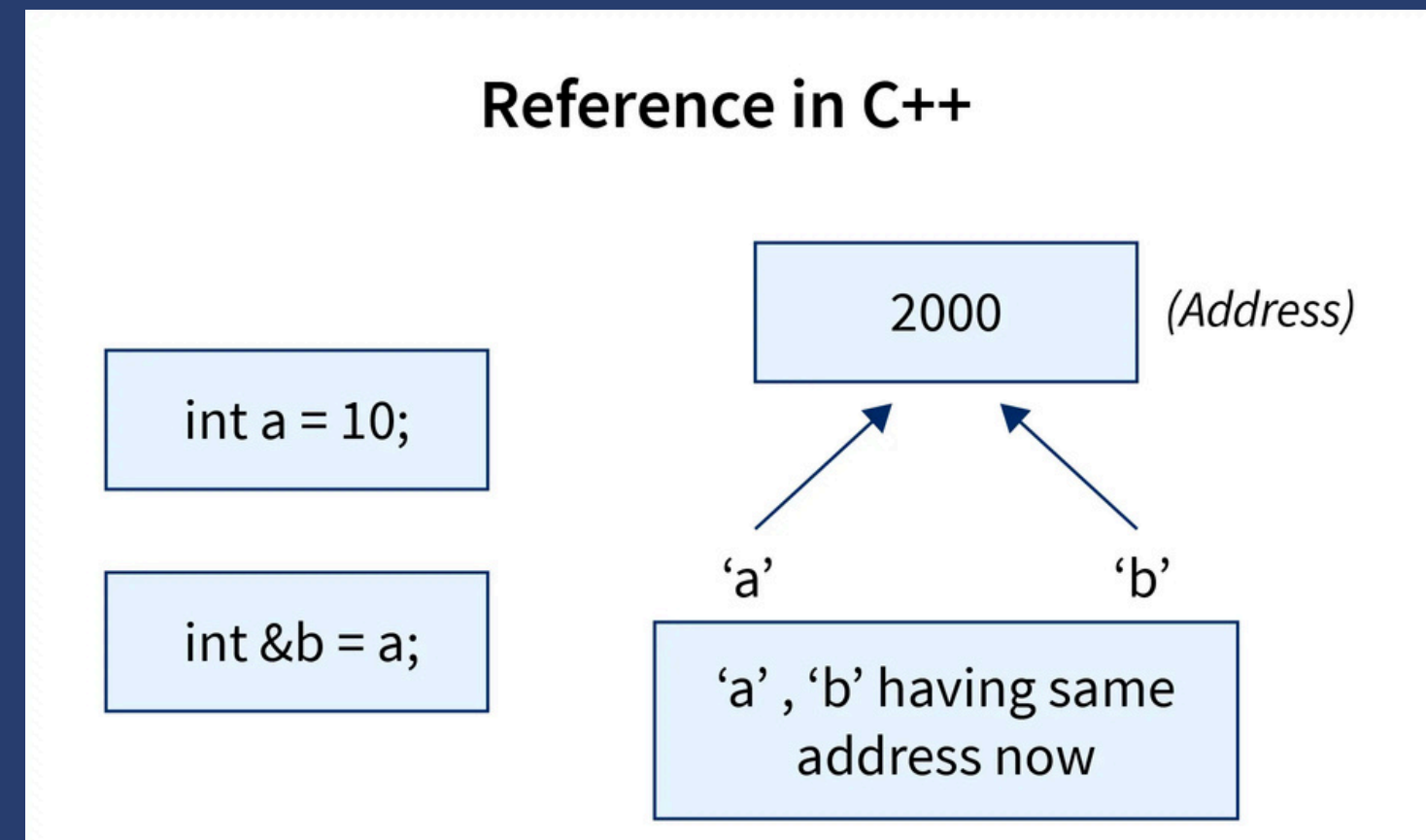
Referanslar



Halihazırda yer alan bir değişkenin **alias**'ıdır.
Orijinal değişkeni direkt olarak kullanabilmeyi ve modifiye edilebilmesini sağlar.

Pointer oluşturma aksine, referans tanımlamak yeni bir değişken oluşturmaz.

İklendirilmiş bir referans, başka bir değişkene referans veremez.



Smart Pointerlar



Pointer'ların otomatik yönetimini sağlayan wrapper sınıflarıdır.

Bunlarla birlikte programcının elle bellek ayırmadan, belleği temizlemeden pointer'ları daha güvenli bir şekilde kullanabilmesi hedeflenir.

Temel iki hali vardır:

- Unique Pointer
- Shared Pointer

`memory` headerında tanımlanmıştır.

Unique Pointer



İşaret ettiğin nesnenin tek sahibi olan akıllı pointer tipidir. Bütün bellek işlemlerini kendisi halleder.

Herhangi bir zamanda tek bir nesnenin sadece bir adet unique pointer sahibi olabilir.

Pointer scope'tan çıktığında işaret ettiği nesneyi otomatik olarak temizler.

Shared Pointer



Unique pointer mekanizmasına benzer şekilde bellek yönetimini halleder fakat bir nesnenin birden fazla sahibi olabilir.

Sahip pointer'ların hepsinin scope'tan çıktığı zaman nesne temizlenir.

Aynı nesneyi gösteren bütün shared pointerlar için ortak bir kontrol bloğu bulunur ve referans sayıları burada takip edilir, sıfıra ulaştığında nesne temizlenir.